When you write the second test, you don't want to replicate the code for `http double`, so you move it outside the `fact`:

```
httpPrototype =
  get: (options, callback) ->
    callback()
    @
  on: (event, callback) ->
    # do nothing

fact 'this test uses a http double', ->
  http = Object.create httpPrototype

fact 'this test also uses a http double', ->
  http = Object.create httpPrototype
```

Even that is tedious. What you really want is to define some setup that's done either before all of your tests or before each of your tests. By now you know that functions are good for this sort of thing. For example, you can extract the double creation into a `createHttp` function to run before each test:

```
createHttp = ->
  http =
    get: (options, callback) ->
      callback()
      @
    on: (event, callback) ->
      # do nothing
  http

fact 'this test uses a http double' ->
  http = createHttp()
  # the rest of the test goes here
```

If you have to create 10 doubles, you'll have to call 10 different setup functions. No thanks. The simplest way to handle setup for groups of tests is to take advantage of function scope. If you have three tests that use an `http double`, put them in a single scope: